ENTREGA TÉCNICA

Pipeline de Dados do MVP CitizenTech

Big Data Architecture & Data Integration

PROJETO

Zela - Zeladoria Urbana por Foto

PAPEL NO MVP — Coletar, tratar e disponibilizar dados para acompanhamento de relatos.

Resumo

A solução implementa um fluxo ETL reproduzível para transformar relatos de zeladoria em dados analíticos. A carga pode ler a tabela **reports** do MVP ou um CSV de demonstração, validar e normalizar registros, separar rejeições e gerar agregações por dia, semana e mês.

O pipeline local foi executado e testado. A DAG **zela_relatos** foi executada no Apache Airflow 2.10.5 via Docker Compose com dados sintéticos; as quatro tarefas finalizaram com sucesso. A execução está documentada nos logs e na captura de tela da página 5.

Entregáveis incluídos

- Código do pipeline ETL e DAG Airflow
- Compose para execução local e integração com banco do MVP
- Diagrama de arquitetura e dicionário de dados
- Testes automatizados: 8 aprovados
- Logs, saídas CSV/JSON e captura da Graph View
- Instruções de reprodução, limitações e cuidados de privacidade

EXECUÇÃO REGISTRADA

40 registros sintéticos lidos; 40 válidos; 0 rejeitados; quatro tarefas Airflow em SUCCESS. Os indicadores são demonstração, não resultados de campo.

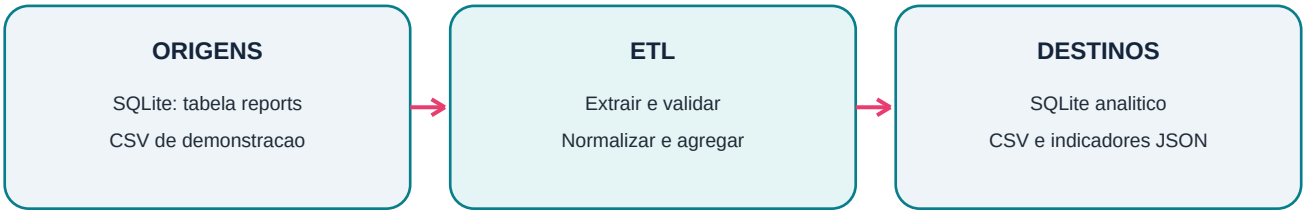
Integrantes da equipe

Joseh Gabriel Trimboli Agra | RM: 553094

Pedro Henrique de Assuncao Lima | RM: 5527476

1. Arquitetura e fluxo

O componente de dados recebe relatos do banco SQLite compartilhado pelo MVP ou de arquivos CSV. A DAG controla as etapas, enquanto o script ETL concentra as regras de validacao e transformacao.



Airflow DAG: preparar fonte -> executar pipeline -> registrar resumo

Etapas do pipeline

Etapa	Responsabilidade
1. Extrair	Ler reports via SQLite ou receber registros CSV; manter identificador report_id.
2. Validar	Conferir campos obrigatorios, datas, categorias, status e coordenadas; registrar motivos de rejeicao.
3. Transformar	Normalizar texto e datas UTC; derivar dia, semana ISO e mes; preservar origem demo/cidadao.
4. Carregar	Recriar tabelas analiticas e gravar registros limpos, rejeitados, agregados e historico de execucao.
5. Orquestrar	DAG Airflow executa as tarefas em ordem e disponibiliza logs por tarefa.

Produtos analiticos

relatos_limpos para consultas por relato; **relatos_rejeitados** para auditoria de qualidade; **agregado_periodo** para contagens e indicadores por categoria, status e origem; **execucoes** para rastreabilidade da carga.

Desenho simplificado: origem -> extracao e validacao -> transformacao -> destino analitico. Os arquivos gerados podem ser inspecionados sem abrir o Airflow.

2. Contrato de dados e privacidade

O contrato comum permite que o aplicativo e as disciplinas usem a mesma chave. A tabela de origem e **reports**; o identificador **id** e exposto como **report_id**.

Camada	Campos e regras principais
Origem reports	id inteiro; created_at UTC; location aproximada; latitude/longitude opcionais; description; category; status; source; campos opcionais da visao computacional e ordem de servico.
relatos_limpos	id, created_at normalizado, dia/semana/mes, local, coordenadas validas, categoria/status/origem canonicos, metadados de visao e indicador de ordem de servico.
relatos_rejeitados	Campos necessarios para corrigir a qualidade, sem imagem ou segredo; coluna motivos lista as validacoes que falharam.
agregado_periodo	periodo_tipo, periodo, category, status, source, total e indicadores de revisao, concordancia e encaminhamento.
execucoes	executado_em, run_id, fonte e indicadores_json para rastrear cada carga.

Regras importantes

• O dado de demonstracao usa source=demo, separado dos relatos cidadaos.

• A DAG nao extrai o nome do arquivo de imagem nem confirm_token.

• A camada analitica nao copia descricao livre; localizacao detalhada e reduzida ao necessario para a analise.

• Coordenadas incompletas ou fora dos limites geograficos sao descartadas na transformacao.

• Os indicadores mostrados nesta entrega foram gerados com dados sinteticos e nao descrevem ocorrencias reais.

O dicionario completo, com tipos, dominios e descricoes de cada coluna, esta no repositorio em **zela_bigdata/DICIONARIO_DE_DADOS.md**.

3. Execução, evidência e reprodução

A execução local do script ETL usou **reports_demo.csv**, um conjunto sintético com 40 registros. A execução Airflow descrita abaixo foi feita com a mesma fonte de demonstração.

40	40	0	40
registros lidos	válidos	rejeitados	marcados demo

Reproduzir a execução local

Na pasta **entrega-bigdata**, execute:

```
python zela_bigdata\dags\zela_pipeline.py --fonte
zela_bigdata\exemplos\reports_demo.csv --saida saida\execucao_local
```

Execução no Airflow

A DAG **zela_relatos** foi executada com Apache Airflow 2.10.5 via Docker Compose. Run ID **manual__entrega_bigdata_20261110**: SUCCESS. As tarefas **extrair**, **validar**, **agregar** e **carregar** finalizaram com sucesso.

A captura da Graph View na página seguinte mostra a execução selecionada em 10/11/2026 às 18:59:47. Logs de tarefa, indicadores e arquivos CSV/JSON estão em **zela_bigdata/evidencias/execucao_airflow/**. A suíte automatizada teve **8 testes aprovados**.

Como interpretar os resultados

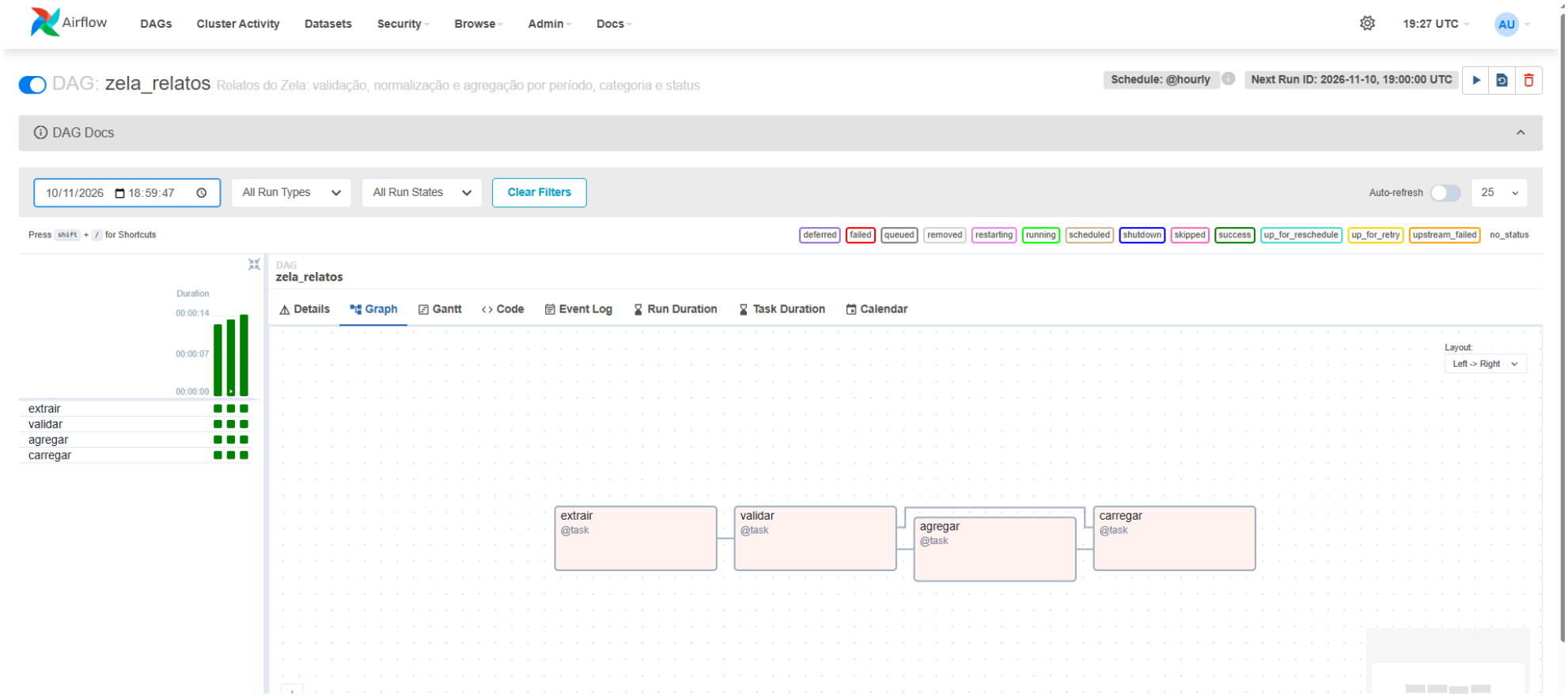
Os 40 registros são sintéticos e marcados como **source=demo**. Taxa de confirmação 0,95; concordância da sugestão de visão 0,974; taxa encaminhada 0,475 são valores de demonstração usados apenas para validar o processamento; não representam ocorrências ou desempenho real.

Repositório e reprodutibilidade

Repositório público da equipe: pasta de entrega na branch entrega-bigdata. O README da pasta contém os comandos do Compose e do pipeline. Os logs e saídas fornecem rastreabilidade para esta execução.

Evidencia visual - Airflow Graph View

DAG zela_relatos | execucao selecionada em 10/11/2026 as 18:59:47



A captura mostra a DAG no Graph View e os estados das tarefas na grade lateral. O log na pagina anterior identifica a execucao manual__entrega_bigdata_20261110; as quatro tarefas finalizaram com sucesso.